

EyeWhere

Smart Glasses for Navigation



Ethan Gray | ergray85@gmail.com

Connor Salpeter | connor.salpeter31@gmail.com

2026-03-09

Abstract

The goal of this project was to develop a functional pair of smart glasses which display navigation information to the user. These glasses, named “EyeWhere”, display navigation information—the direction, distance, and street name of the next turn—in the peripheral of the user’s vision. This enables the user to follow directions without looking away from the road, a safer alternative to typical navigation devices which require the user to look down at a mobile phone or dashboard display.

Navigation information is generated on a PC through a Python script which queries the Valhalla Turn-by-Turn Route API running inside a Docker container built from OpenStreetMap data. The script communicates with the Valhalla service by sending and receiving JSON over a locally exposed port. The user’s coordinates, given in decimal degrees, can update in real time over a USB GPS module connected to the PC, resulting in real-time generation of directional commands.

These generated directional commands are encoded into a custom 14-byte packet, and are sent to the smart glasses over Bluetooth Low Energy (BLE) using the bleak library. The glasses contain an RN4871 BLE module that establishes a connection to the PC and forwards the received byte packets to the on-board MCU (MSP430) over UART. The BLE device is configured and made to advertise through firmware commands sent from the MCU when the smart glasses power on. The MCU parses the received byte packets to ensure they are valid, then decodes the packet and updates the Micro-OLED display accordingly over I2C.

The graphics displayed on the micro-OLED display pass through the optical system, consisting of a convex lens and a semi-reflective film, delivering the image to the user’s eye with suitable magnification and focused near-infinity.

The mechanical body of the smart glasses themselves were designed in CAD as three primary parts—the left arm, the right arm, and the frame—and fabricated through 3D printing. The body was designed to house all components, including the battery, OLED display, optical assembly, and electronics, while maintaining the appearance and functionality of a conventional pair of glasses, including foldable arms. The MCU, BLE module, power regulation circuitry, and supporting electronics were integrated onto a custom PCB mounted on the outer wall of the glasses’ left arm.

1. Objectives	4
2. Concept and Rationale	5
3. Overview of Functional Requirements	7
4. Functional Component #1 - Firmware	11
4.1 Approach and Design	11
4.2 Inputs and Outputs	13
4.3 Parameters	14
4.4 Testing and Results	15
5. Functional Component #2 - Software	16
5.1 Approach and Design	16
5.2 Inputs and Outputs	18
5.3 Parameters	19
5.4 Testing and Results	20
6. Functional Component #3 - Optics	24
6.1 Approach and Design	24
6.2 Inputs and Outputs	28
6.3 Parameters	28
6.4 Testing and Results	28
7. Functional Component #4 - Mechanical Design	30
7.1 Approach and Design	30
7.2 Inputs and Outputs	34
7.3 Parameters	34
7.4 Testing and Results	35
8. Functional Component #5 - Electronics Design	35
8.1 Approach and Design	35
8.2 Inputs and Outputs	38
8.3 Parameters	40
8.4 Testing and Results	40
9. System Evaluation	43

1. Objectives

Our goal was to design glasses that can project images into the corner of the wearer's vision without being overbearing and conspicuous. We wanted a discrete product that feels polished and functional—something that we would actually want to wear. Referring back to our proposal, we managed to deliver everything that we intended and followed through with all of our plans. Specifically, we

- designed a custom PCB to carry out the function of receiving data wirelessly and powering/updating the screen
- wrote firmware to interface with our BLE module to receive and parse custom data packets and update the screen with bit-banged I2C
- wrote software to generate navigation instructions based on starting and ending coordinates and send instructions over BLE
- selected suitable optical components and integrated them into the assembly
- designed and manufactured a glasses assembly that houses the optics and electronics.

The thing that we did not accomplish is implementing the software side on a mobile app for improved utility. Currently, the software runs on our laptops, which is not practical if you want to use the glasses for their intended purpose. To use them, you have to have your laptop with you and running the software.

2. Concept and Rationale

The goal of this design project was to design and build a functional pair of smart glasses which display navigation information to the user. This enables the user to follow directions without looking away from the road. The navigation information is displayed in the peripheral of the glasses, as shown in Fig. 2.1.



Figure 2.1 – Concept Sketch Depicting Navigation Information Displayed on the Peripheral of a Lens

This project is interesting because it incorporates elements of electronics design, firmware and software design, and optics — all of which we wanted to incorporate in our project. The development of these glasses is a major safety improvement for people trying to use maps while they bike or drive. In addition to this application, we have also thought of many other possible applications of our glasses that could be implemented with different software and firmware that could be impactful, such as use in manufacturing facilities by operators in their safety glasses. Other smart glasses exist on the market, but are very expensive and have lots of functionality, such as text messaging and internet access, which are major distractions on the road.

The thing that was the most important to us throughout the project was developing the mechanical design and electronics as much as possible and making our design look and feel like a normal pair of glasses. We wanted to tuck away the optical assembly, battery, and PCB so that they do not interfere with the product, and also have a good quality image that is superimposed in the wearer's peripheral vision without impeding their field of view or visual clarity. The firmware and software were necessary to make the project successful, but in the end we take the most pride in the physical product that we created. The main metrics of success are:

- Wearability
- Image clarity
- Electronics design reliability, features and form factor
- BLE reliability, firmware and software design quality

- Form factor and product design

Early in the project we decided to make the most polished aspect to be firmware and hardware — that is the PCB design and the firmware to run on the MCU for BLE communication and updating the screen. We chose this because it presented less risk than other subsystems like the optics. We believe that our implementation of the firmware and hardware was especially successful. Our firmware was robust, both with regards to communicating over BLE as well as updating the display, and our custom PCB worked exactly as intended.

We accomplished almost everything that we set out to do. The element that is missing that would tie everything together is a mobile app for generating the navigation commands using the GPS coordinates available and sending the information to the glasses. Currently we have all of that implemented to run on our laptops because we are more familiar with the development environment there.

The most critical module in this project is the optical system, which is described in detail in Section 6. This module was identified as the most critical due to its high complexity and risk, primarily stemming from the precise alignment and fine mechanical adjustments required to properly focus and collimate the image wavefront. Small errors in alignment can significantly degrade the image quality or prevent the image from reaching the user's eye, which would cause the system to fail. Additionally, many commercial smart glasses use advanced optical manufacturing techniques, such as nano-scale waveguide fabrication, which are way beyond the scope of this project. As a result, our design was constrained by feasibility and available optics, which introduced additional design challenges. However, the scope of our optical system was intentionally simplified compared to professional grade solutions, and a clear design and development plan was created and executed, as outlined in Section 6.

The data infrastructure is as follows:

- Navigation information is generated on a PC through a Python script which queries the Valhalla Turn-by-Turn Route API running inside a Docker container built from OpenStreetMap data. The script communicates with the Valhalla service by sending and receiving JSON over a locally exposed port. The user's coordinates, given in decimal degrees, can update in real time over a USB GPS module connected to the PC, resulting in real-time generation of directional commands.
- These generated directional commands are encoded into a fixed-length 14-byte packet, and are sent to the smart glasses over Bluetooth Low Energy (BLE) using the bleak library. The glasses contain an RN4871 BLE module that establishes a connection to the PC and forwards the received byte packets to the on-board MCU (MSP430) over UART. The BLE device is configured and made to advertise through firmware commands sent from the MCU when the smart glasses power on.
- The MCU parses the received byte packets to ensure they are valid, then decodes the packet and updates the Micro-OLED display accordingly over I2C.

We have different software versions that can send data for testing the glasses, which is what we showed during the demonstration. The final version of the firmware is robust, and works with all of our various test and production software packages.

3. Overview of Functional Requirements

The functional requirements of this project are summarized in the following table.

Table 3.1 - List of Functional Components

Functional Requirement	% Effort
FC#1: Firmware	25
FC#2: Software	5
FC#3: Optics	20
FC#4: Mechanical Design	25
FC#5: Electronics Design	25

A brief description outlining the purpose and scope of each functional component can be seen below.

1. Firmware:

The purpose of the firmware functional requirement was to control the smart glasses' electronics and convert received navigation data into visual instructions for the user. Its scope included BLE module setup, serial data reception, packet parsing, and OLED display control.

Firmware was developed on the MSP430 microcontroller to communicate with the RN4871 BLE module over UART and the SSD1315-based micro-OLED over I2C. At startup, the MCU configures the BLE module and commands it to advertise. Once connected to the PC, the BLE module forwards incoming navigation bytes to the MCU through UART. The firmware buffers these bytes, identifies valid 14-byte packets, and decodes the encoded navigation information, including turn direction, distance, units, street name, and any special message.

Firmware was also developed to initialize and control the OLED. After decoding a valid packet, the MCU updates the display with the appropriate symbols and text, such as directional arrows and distance information, using bit-banged I2C routines. This firmware therefore served as the interface between wireless communication from the PC and the user facing display.

2. Software:

The purpose of the software functional requirement was to generate navigation commands from GPS position data and transmit them to the smart glasses in a custom digital format. Its scope included processing user coordinates, determining the next navigation instruction, encoding the resulting information into a custom packet structure, and sending the packet wirelessly to the glasses.

Python code running on a PC was used to convert an initial and destination coordinate pair into turn-by-turn instructions. From the current navigation state, the software determines the next required

instruction, such as turn direction, remaining distance, and the street name. These commands can be recalculated in real time as the user moves according to live GPS updates.

The generated navigation instruction is encoded into a custom byte packet and transmitted to the smart glasses for interpretation by the firmware. This functional requirement therefore served as the bridge connecting location data to user-facing navigation output.



Figure 3.1 – Navigation User Interface

3. Optics

The purpose of the optics functional requirement was to deliver the graphical output of the micro-OLED display to the user's eye in a clear and readable way.

A convex lens was placed between the display and the user's eye to adjust the wavefront curvature so that the virtual image appears focused near infinity. Without this optical element, the display appears out of focus and is impossible to read because it is located too close to the eye. The effect of the convex lens can be seen in Figure 3.2.

Since the display is mounted in the arm of the glasses and faces forward, a semi-reflective film applied to the left lens acts as an optical combiner. Light from the display passes through the convex lens and reflects from the film into the user's eye while still allowing the user to see the surrounding environment. The display and lens assembly were positioned at a fixed angle so that the reflected image enters the user's pupil and appears in the peripheral field of view.



Figure 3.2 – Demonstration of the Effect of the Convex Lens for Close Distance Viewing. Left: No Lens, Right: with Convex Lens

4. Mechanical Design

The purpose of the mechanical design function requirement was to create a pair of glasses that houses all of our parts and systems without imposing on the natural look and feel of glasses. Fig. 3.3 has the assembly diagram, showing how the various parts are designed for function but still resemble the corresponding parts on regular glasses. The battery is housed in the right arm, and the left arm houses the optical assembly and the PCB.

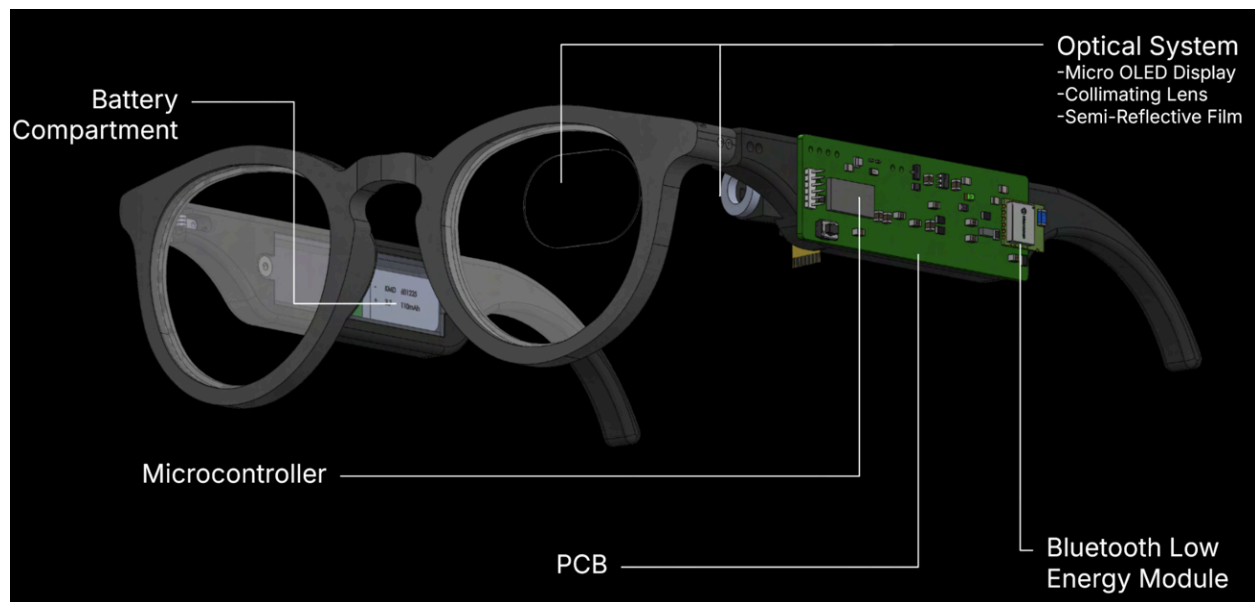


Figure 3.3 – Full Assembly Diagram

5. Electronics Design

The purpose of the electronics design functional requirement was to deliver the electrical demands in a way that could be implemented cleanly on the glasses. A custom PCB was developed in Altium Designer

with the exact functions and features that were required. The board layout was chosen with the mechanical design in mind, striving to fit the profile of the arm.

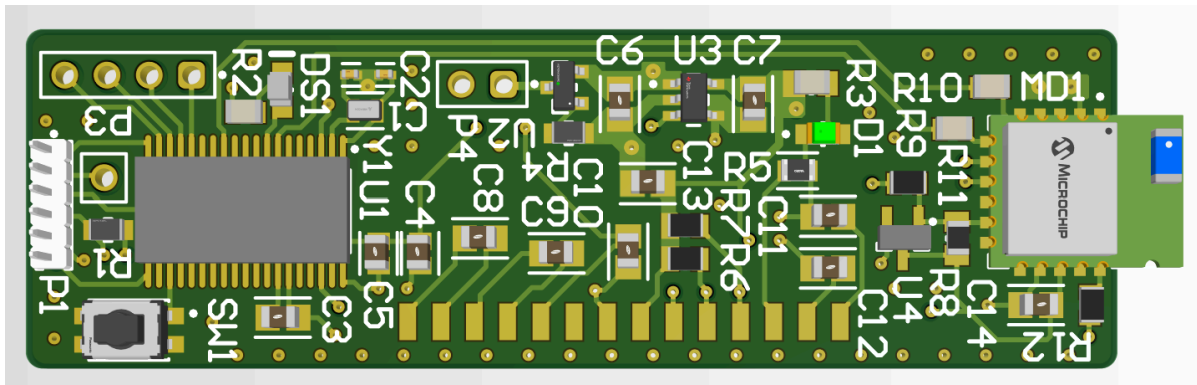


Figure 3.4 – PCB Top Side

4. Functional Component #1 - Firmware

4.1 Approach and Design

The objective of the firmware functional requirement was to set up the MCU to receive navigation data from the BLE module and update the OLED display in real time. Firmware was developed on the MSP430 microcontroller to initialize the hardware, configure the RN4871 Bluetooth module, buffer and parse received packets, and control the SSD1315-based OLED display.

The firmware was organized around three main tasks: initialization of the microcontroller peripherals, communication with the BLE module over UART, and control of the OLED display through software-implemented I2C. A high-level overview of the firmware structure is shown in Figure 4.1.1.

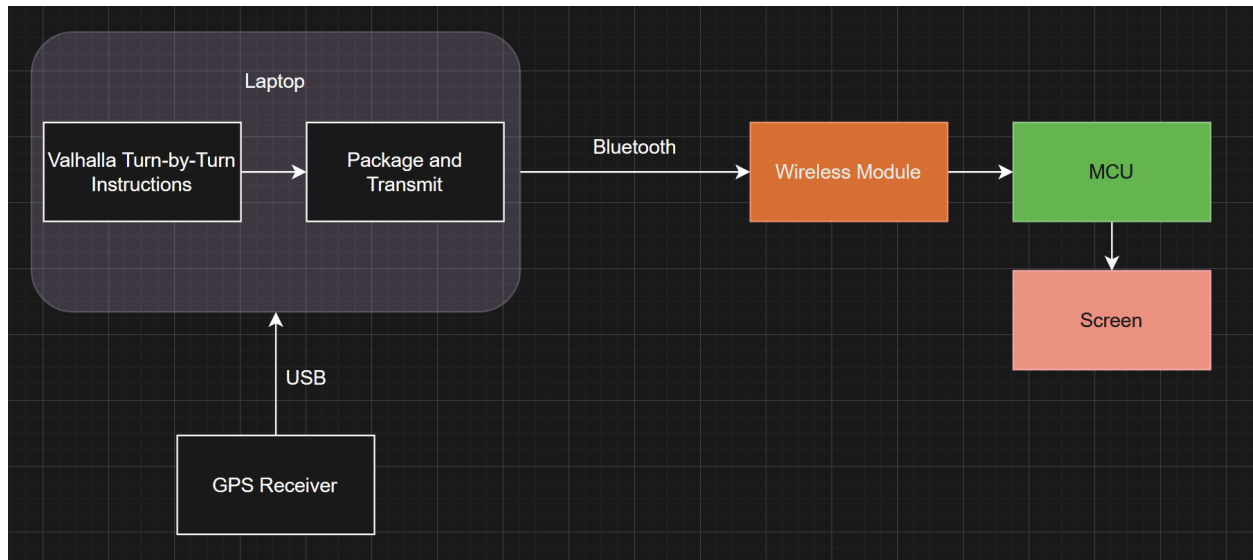


Figure 4.1.1 – High-Level Data Infrastructure

The RN4871 is configured over UART by placing it into command mode by sending “\$\$\$” from the MCU to the BLE module, then sending the required ASCII configuration commands, and then setting it up in advertising mode. In its final operating state, the BLE module functions as a transparent UART bridge, forwarding all received BLE bytes directly to the MSP430.

Whenever a byte is received from the RN4871, the BLE UART receive interrupt is triggered, and the byte is immediately copied into a circular capture buffer. This allows the interrupt service routine to complete quickly without spending time parsing packets or updating the display. The contents of this circular buffer are checked in the main loop, where valid 14-byte packets beginning with the defined start byte are identified. If the incoming bytes do not form a valid packet, they are discarded. If a valid packet is identified, it is copied into a second buffer of accepted packets.

Once a valid packet is available, the firmware decodes the direction, distance, unit, street name, and special-message fields, and converts them into display content. The OLED is controlled through a bit-banged I2C interface implemented in firmware.

To implement I2C, the SDA and SCL lines were configured to behave as open-drain signals using GPIO pins and external pull-up resistors. When idle, each pin is configured as a high-impedance input, allowing the pull-up resistor to hold the line high. To drive either line low, the corresponding pin is reconfigured as a digital output and driven low. This way, the MSP430 does not actively drive the line high, it either pulls the line low or releases it, which matches the open-drain behavior required for I2C communication.

I2C signaling was generated in software by directly controlling the timing of the SDA and SCL lines. Following the SSD1315 datasheet, firmware routines were written to generate the START and STOP conditions, transmit individual bits (making up either the slave address, command bytes, or pixel data bytes), and read the acknowledge bit returned by the display driver (SSD1315). A START condition is generated by pulling SDA low while SCL remains high, after which each data bit is set on SDA while SCL is low. After eight bits have been transmitted, the SDA line is released so that the OLED driver can pull it low to acknowledge that the byte was received. Communication is terminated by issuing a STOP condition, which happens when SDA transitions high while SCL is high. This timing sequence, based on the SSD1315 datasheet, is shown in Figures 4.1.2 and 4.1.3.

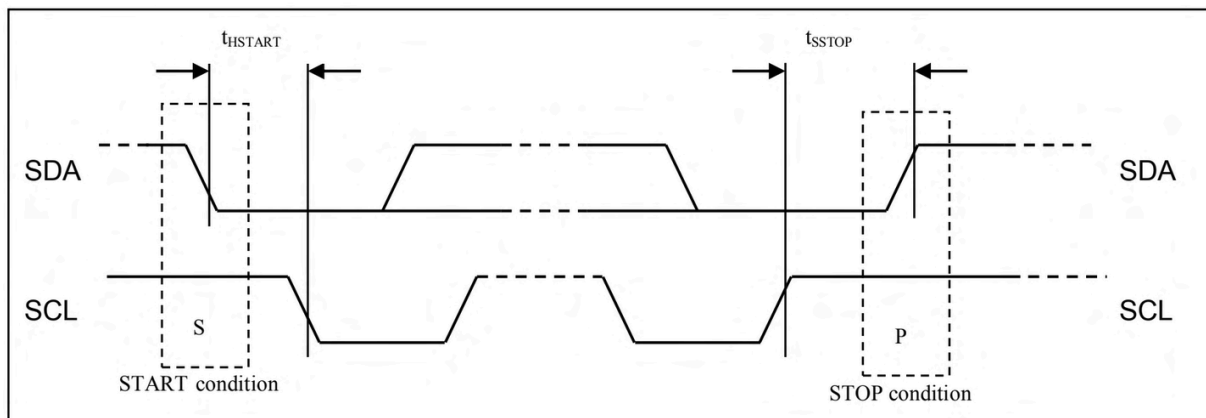


Figure 4.1.2 – I2C Timing Sequence Outlined in the OLED Driver Datasheet

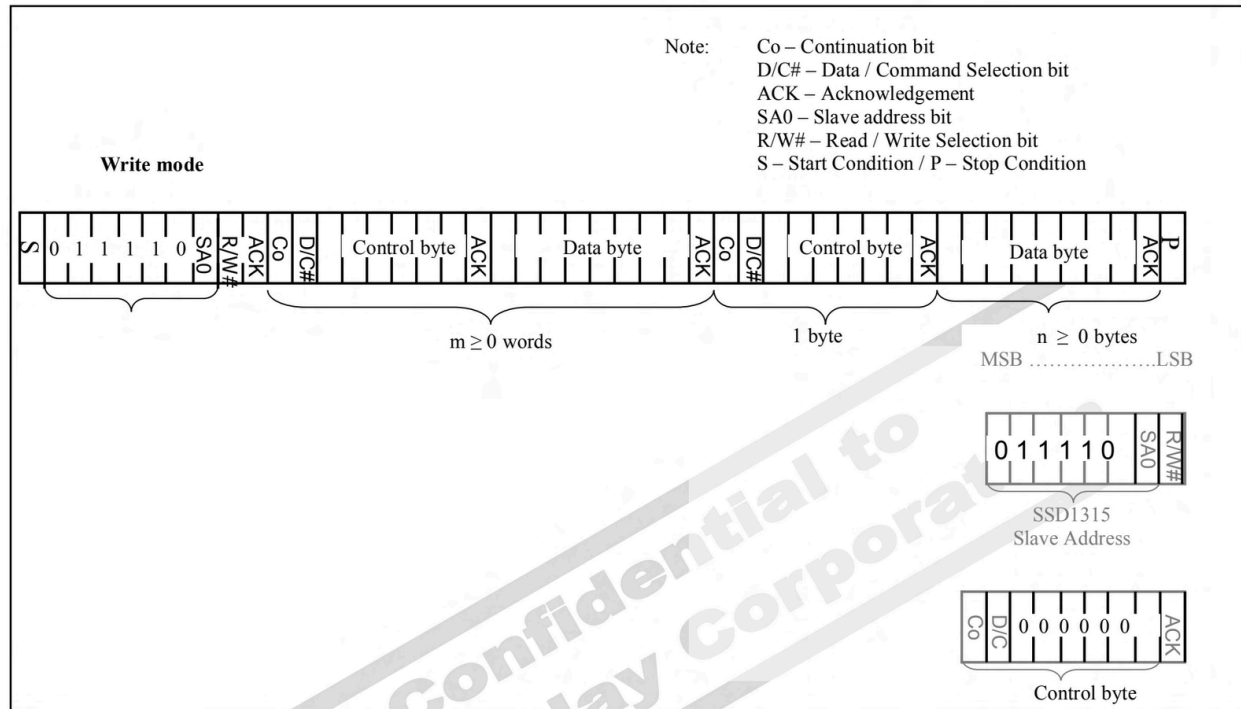


Figure 4.1.3 – I2C Byte Packet Structure Outlined in the OLED Driver Datasheet

Using the SSD1315 datasheet, the required device address, control bytes, initialization sequence, and page-addressing method were identified and implemented. Custom display routines were then written to initialize the OLED, draw bitmap icons, and render text using a font table.

To ensure reliable I2C communication, testing was performed on the line timing and pull-up resistor values. These parameters affected the rise and fall times of the SDA and SCL lines through their RC time constant. The final timing and resistor values were selected experimentally to ensure repeatable and reliable communication with the SSD1315 driver.

4.2 Inputs and Outputs

The inputs are packets sent by FC2 in the serial message format specified in Section 5. The output from the MCU is a sequence of bytes sent over I2C that update the screen. Figure 4.2.1 and 4.2.2 show the register map for the pixel data, where each byte represents one column of eight rows. Each pixel is binary and the memory pointer is incremented automatically after each write. To write to a new region of the screen we used move commands (for example writing the next character in a string), and we used arrays stored in the code memory to retrieve the pixel data for various characters and images.

		Row re-mapping
PAGE0 (COM0-COM7)	Page 0	PAGE0 (COM 63-COM56)
PAGE1 (COM8-COM15)	Page 1	PAGE1 (COM 55-COM48)
PAGE2 (COM16-COM23)	Page 2	PAGE2 (COM47-COM40)
PAGE3 (COM24-COM31)	Page 3	PAGE3 (COM39-COM32)
PAGE4 (COM32-COM39)	Page 4	PAGE4 (COM31-COM24)
PAGE5 (COM40-COM47)	Page 5	PAGE5 (COM23-COM16)
PAGE6 (COM48-COM55)	Page 6	PAGE6 (COM15-COM8)
PAGE7 (COM56-COM63)	Page 7	PAGE7 (COM 7-COM0)
	SEG0 -----SEG127	
Column re-mapping	SEG127 -----SEG0	

Figure 4.2.1 – GDDR4M Pages Structure

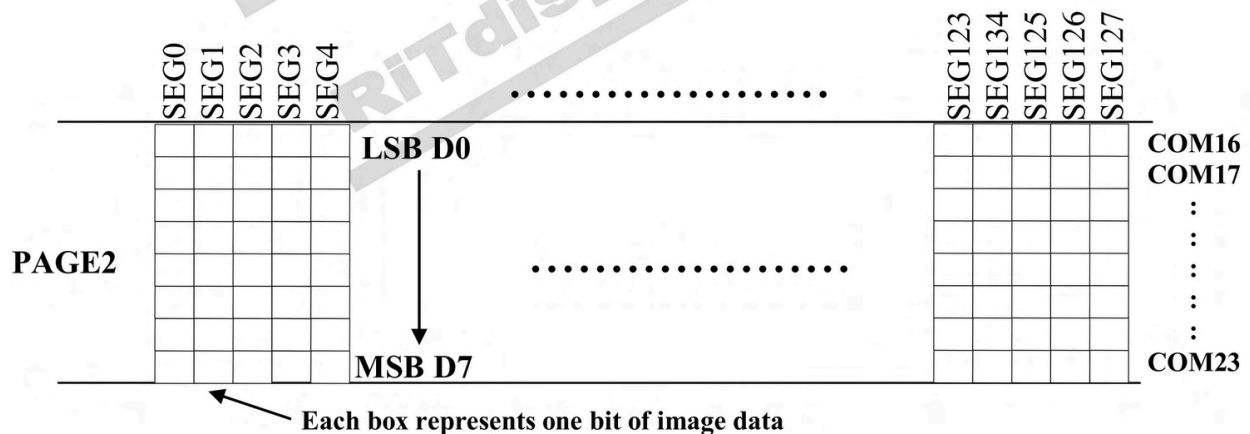


Figure 4.2.2 – GDDR4M Pages Structure

The graphics displayed on the screen include the next turn direction, distance before the turn, and the name of the street to turn on. We did not have enough pixels to display long street names, so we capped it at eight characters.

4.3 Parameters

The main parameters for this FC are the wireless data transmission rate and the data transmission rate between the MCU and the screen, which determines the maximum refresh rate of the screen. The wireless data transmission rate for receiving the information packets is not a bottleneck, and is not important in this discussion. What is the most important is the data transmission rate between the MCU and the screen, which is the bottleneck for the framerate that we are able to achieve. From the display data sheet:

The display clock (DCLK) for the Display Timing Generator is derived from CLK. The division factor “D” can be programmed from 1 to 16 by command D5h

$$DCLK = F_{osc} / D$$

The frame frequency of display is determined by the following formula.

$$F_{FRM} = \frac{F_{osc}}{D \times K \times \text{No. of Mux}}$$

where

- D stands for clock divide ratio. It is set by command D5h A[3:0]. The divide ratio has the range from 1 to 16.
- K is the number of display clocks per row. The value is derived by $K = \text{Phase 1 period} + \text{Phase 2 period} + K_0 = 2 + 2 + 99 = 103$ at power on reset (i.e. $K_0 = 99$). Please refer to **Section 6.6** for the details of the “Phase”.
- Number of multiplex ratio is set by command A8h. The power on reset value is 63 (i.e. 64MUX).
- F_{osc} is the oscillator frequency. It can be changed by command D5h A[7:4]. The higher the register setting results in higher frequency.

Figure 5.3.1 – Screenshot from driver datasheet with framerate calculation
(<https://www.buydisplay.com/download/ic/SSD1315.pdf>)

With the I2C speed that we converged on, we ended up at about 1FPS for no tearing. The rate is slow because of the quantity of commands and data that we write to the screen for each frame, and could be improved by speeding up the I2C. We deemed that this was not important, because 1FPS is fast enough for portraying navigation instructions.

4.4 Testing and Results

The development of the firmware followed these steps:

1. Complete a functioning version of the firmware that enables us to set pixels on the screen.
2. Write a function that uses the character pixel arrays to determine what the pixel grid should be for an entire frame. This involves spacing all of the characters and having the correct offsets for each new character.
3. Write a function to parse data packets and determine the arrow direction, distance number and unit, and the street name.
4. Develop the firmware to interface with the wireless module and test that packets can be received wirelessly.
5. Integrate (4) with (1-3) and test with the software described in Section 5.

For the first two steps we tested our initial firmware for the screen that wrote predetermined messages and verified that the messages were correct. After that was working we implemented parsing functions and tested with the C# interface shown in Fig. 5.4.1 using the 14-byte packets that we devised for this application.

The firmware used to set up the BLE module was initially developed as a standalone subsystem. The objective of this code was to establish a Bluetooth connection with Microchip's SmartData application, which allows text to be transmitted and received over BLE. This required sending ASCII commands to the RN4871 module over UART to enter command mode, configure its operating parameters, and enable advertising. Whenever a command was sent to the BLE module, the module returned a response message. The firmware was designed to forward these responses to the PC over a second UART interface, letting us verify that it was working. As a result, one UART channel was used for communication between the MSP430 and the RN4871 module, while the second UART channel was used for communication between the MSP430 and the PC. Once valid responses were observed and the command to enable advertising was issued, the RN4871 appeared as an available device in the SmartData application. This confirmed that the firmware successfully configured the module and established BLE functionality.

5. Functional Component #2 - Software

5.1 Approach and Design

The objective of the software functional component was to generate real-time navigation instructions from GPS coordinates and transmit these instructions to the smart glasses over a Bluetooth Low Energy (BLE) connection. The software is responsible for computing navigation routes, extracting directional commands, encoding those commands into a compact packet format, and maintaining a wireless connection with the glasses.

The software system is implemented primarily in Python and runs on a PC. A high-level block diagram of the software architecture is shown in Figure 5.1.1.

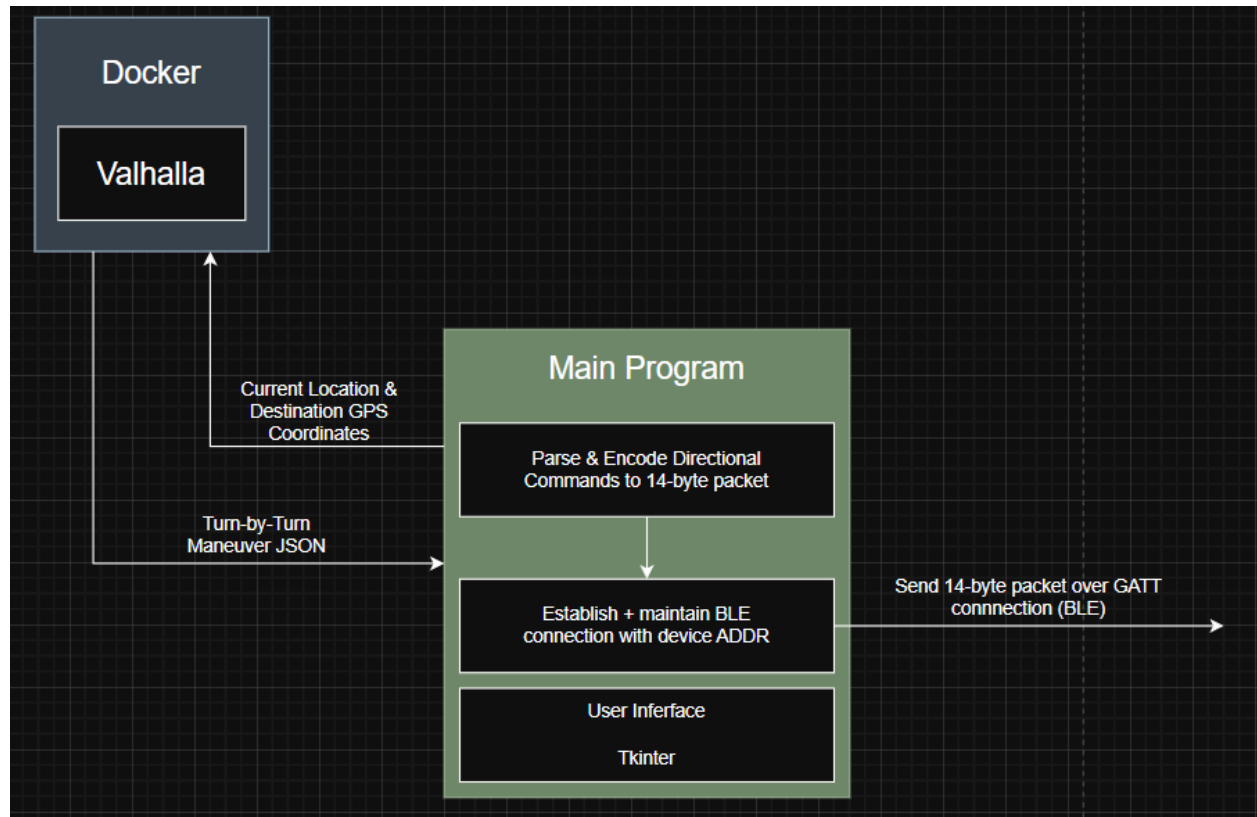


Figure 5.1.1 – High Level Block Diagram of the Software System Architecture

The navigation route generation is handled by the Valhalla routing engine, which runs inside a Docker container on the host computer. Valhalla uses OpenStreetMap data to compute turn-by-turn navigation routes between two sets of GPS coordinates. The Python program communicates with the Valhalla server through HTTP requests sent to a local network port exposed by the Docker container.

When the user specifies a starting location and destination, the Python program sends the corresponding GPS coordinates to the Valhalla routing API. The API returns a JSON response containing detailed route information, including distance segments, maneuver types, and street names. The program parses this response to extract the next navigation instruction, such as turning direction and distance to the maneuver.

Once the navigation instruction is extracted, it is converted into a compact binary representation suitable for transmission to the smart glasses. A custom 14-byte packet format was designed to encode the information required by the microcontroller. This packet contains fields representing the turn direction, distance to the maneuver, distance units, street name characters, and special status flags such as arrival notifications.

After the packet is generated, the Python program transmits it to the glasses using a Bluetooth Low Energy connection. Communication is implemented using the Bleak Python BLE library, which provides an interface for interacting with BLE devices. The program establishes a connection with the RN4871 BLE module integrated in the glasses and sends the encoded packet through a GATT characteristic write operation.

To ensure reliable communication, the program continuously maintains the BLE connection and periodically sends keep-alive packets to prevent the connection from timing out. Once connected, the smart glasses receive the packet and forward it to the onboard microcontroller via UART, where it is decoded and displayed on the micro-OLED.

A simple graphical user interface was also implemented using the Tkinter Python library. This interface allows the user to enter start and destination coordinates, initiate route calculations, and monitor the status of the BLE connection. The interface also displays debugging information such as transmitted packets and connection status.

Overall, the software system integrates route generation, packet encoding, and wireless communication into a single pipeline that continuously converts navigation data into display instructions for the smart glasses.

5.2 Inputs and Outputs

The navigation software serves as the interface between the route-generation system running on the host device and the smart glasses. Its primary function is to convert navigation data into a compact packet format suitable for wireless transmission.

The inputs to the software consist of the user's destination and the user's current geographic coordinates, which can be obtained automatically from the host device through a connected GPS module. These coordinates are used to request navigation routes from the routing engine. The routing engine returns turn-by-turn route information, including the direction of the next maneuver, the distance to the maneuver, and the associated street name.

The software parses this information and encodes the relevant navigation instruction into a Bluetooth Low Energy (BLE) data packet, which is transmitted to the smart glasses. BLE was selected because it provides low-power wireless communication and is natively supported on most laptops and mobile devices.

The output of the software module is therefore a 14-byte navigation packet that contains all information required by the microcontroller in the smart glasses to update the display. The packet structure is shown below.

Byte 1 – Start Byte:

Used to indicate the start of a new packet and to validate that a packet is received and aligned in firmware. This byte always has the value 255.

Byte 2 – Direction Byte:

Encodes the direction of the next maneuver (e.g., left turn, right turn, U-turn, lane change).

Bytes 3–4 – Distance Bytes:

Represent the distance to the next maneuver. The first byte encodes the hundreds and thousands digits, while the second byte encodes the tens and ones digits. Since the value 255 is never required in this encoding scheme, it can be reserved exclusively for the start byte, eliminating the need for an escape or correction byte.

Byte 5 – Unit Byte:

Indicates whether the distance is expressed in meters or kilometers. When kilometers are selected, the first distance byte represents the ones, tens, and hundreds digits, while the second distance byte represents the tenths and hundredths digits, allowing distances up to 254.99 km.

Bytes 6–13 – Street Name Bytes:

Contain the street name associated with the next turn. Each byte encodes a single ASCII character, which allows up to 8 characters. Longer street names will be truncated to fit this limit.

Byte 14 – Special Message Byte:

Used to convey special alerts or messages, such as a wrong turn notification, arrival at your destination, an accident warning, or a speed-trap warning.

5.3 Parameters

The navigation software contains less parameters compared to the hardware modules in the system. Most of the software logic is straightforward once the navigation instructions have been generated. However, a few parameters were defined during development that affected the reliability of the wireless connection and the readability of the navigation information displayed on the smart glasses.

One configurable parameter is the Bluetooth device address of the RN4871 module integrated in the smart glasses. When the Python program establishes a Bluetooth Low Energy (BLE) connection, it must connect to the correct device among any other nearby Bluetooth devices. During development, a separate Python script was written to scan for the unique MAC address of the RN4871 module. This process needed to be performed for each RN4871 module used during testing, and the script made it straightforward to identify the correct address. Once identified, the MAC address was stored in the software configuration so that the program could automatically connect to the intended device.

Another important parameter was the BLE transmission and keep-alive interval used to maintain the wireless connection. The RN4871 module requires periodic communication to prevent the BLE connection from timing out. When no navigation updates are available, the software therefore sends periodic keep-alive packets to maintain the connection. The keep-alive interval was selected to be frequent enough to ensure that the connection remains active, while minimizing unnecessary wireless transmissions in order to reduce power consumption and communication overhead. This interval was optimized experimentally by monitoring the power consumption of the device and testing the behavior of the BLE connection over long periods of inactivity. For example, commands were transmitted after extended idle periods (e.g., after 20 minutes of connection time) to verify that the connection remained active. Through this testing process, the keep-alive interval was adjusted until the BLE connection could

be maintained indefinitely without producing a measurable increase in average power consumption compared to the case with no keep-alive packets.

5.4 Testing and Results

To verify the functionality of the software module, testing was performed incrementally, beginning with isolated subsystems and ending with full integration of routing, packet generation, BLE communication, and display output. This allowed us to validate each stage of the software before combining them into the complete system.

Test 1: Manual OLED Update Through Direct Packet Transmission

The first software test verified that correctly formatted 14-byte packets produced the intended display output on the OLED. A C# Windows Forms application was developed to emulate the OLED screen, preview the packet contents, and transmit manually entered packets directly over UART. This allowed the direction, distance, units, street name, and special-message fields to be tested independently of the routing and Bluetooth software. Successful operation was confirmed when the OLED emulator output matched the transmitted packet contents and the physical display updated as expected.

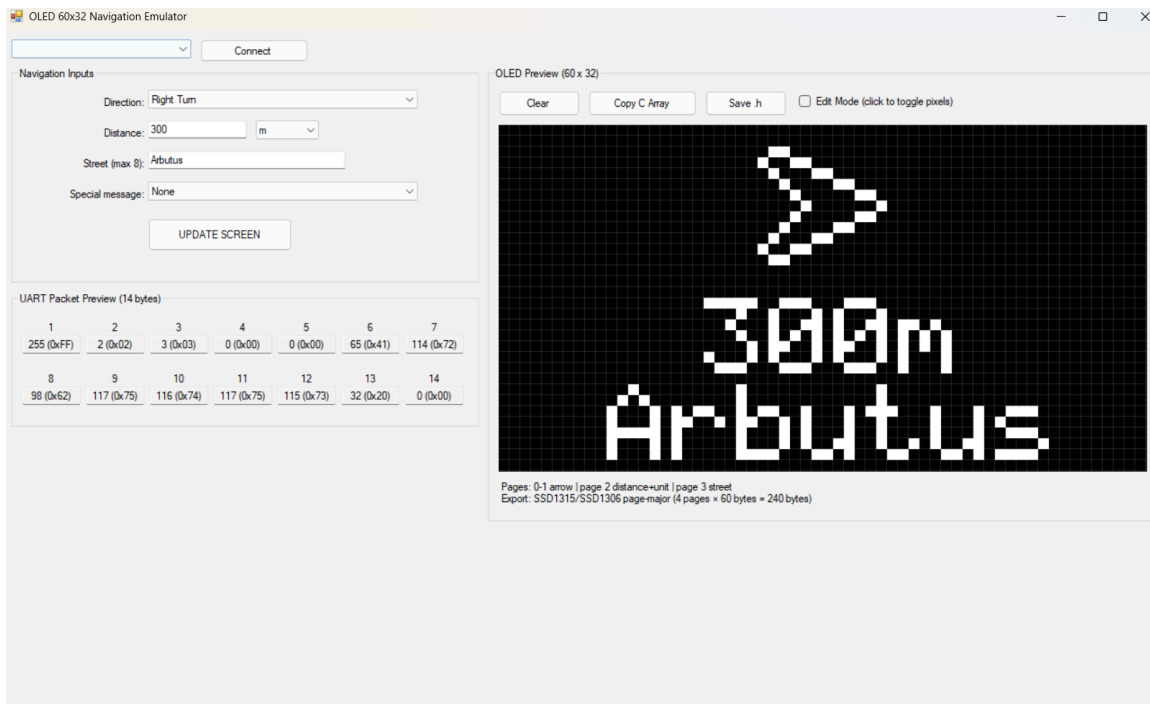


Figure 5.4.1 – Windows Forms App Made to Update the OLED Display Directly over UART

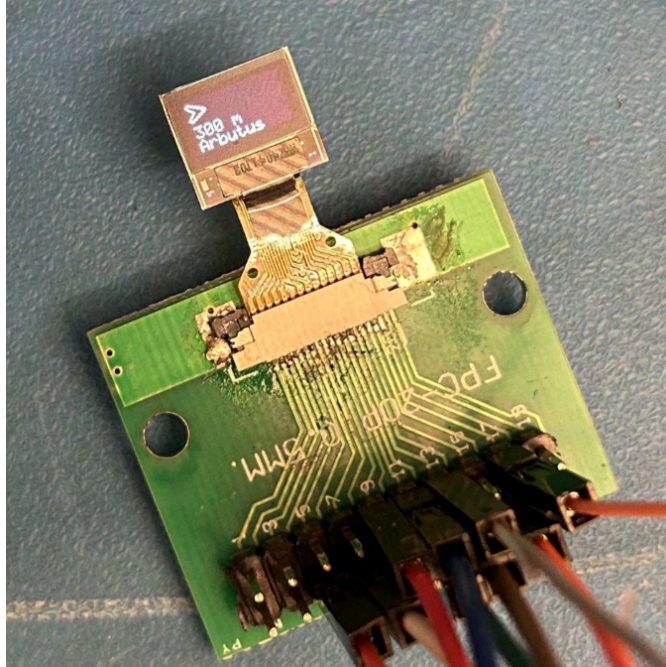


Figure 5.4.2 – Test Result from Sending Data with the Windows Forms App, note that making the text centred was done after this photo was taken

Test 2: Route Generation Using Valhalla Only

The second test verified that the routing engine produced the expected maneuver data before any integration with packet encoding or Bluetooth transmission. A standalone Python script was written to send origin and destination coordinates to the routing engine and print the returned maneuver information, including turn direction, distance, and associated street name. This confirmed that routes could be successfully generated and that the required navigation fields could be extracted from the returned route data.

```
--- Next 1 ---  
Direction: left  
Distance: 347 m  
Street: West 12th Avenue  
--- Next 2 ---  
Direction: left  
Distance: 793 m  
Street: Burrard Street  
--- Next 3 ---  
Direction: left  
Distance: 1.2 km  
Street: West 4th Avenue  
--- Next 4 ---  
Direction: right  
Distance: 76 m  
Street: Larch Street  
--- Next 5 ---  
Direction: unknown(5)  
Distance: 0 m  
Street: Your destination is on the right.
```

Figure 5.4.3 – Output for a Route Generation Test using Valhalla

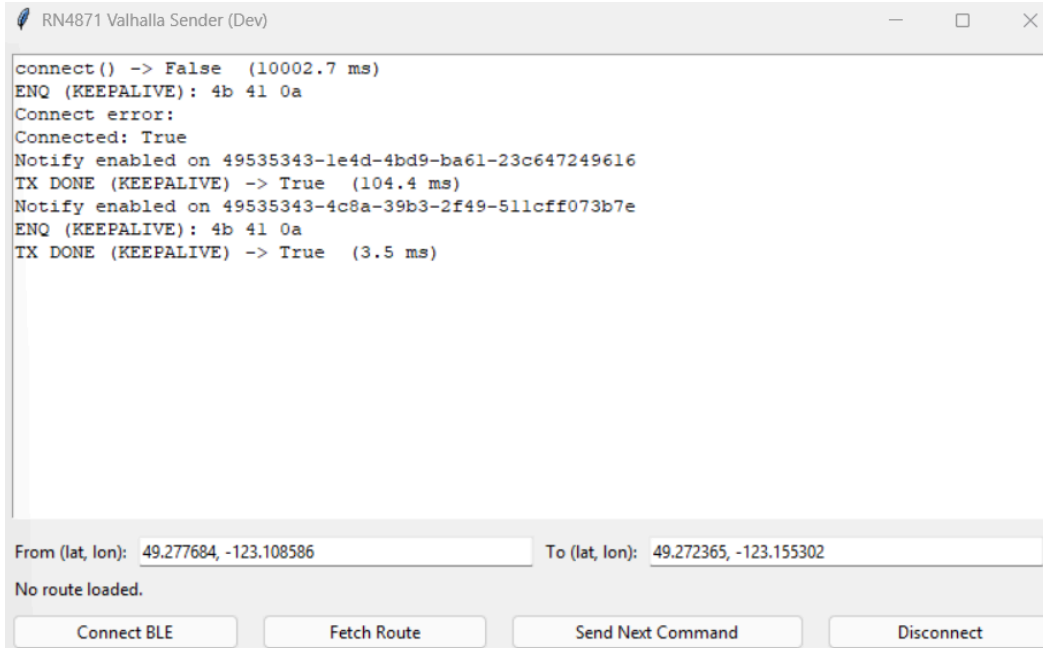
Figure 5.4.3 shows output for a sample route generation test. In that sample test, the starting location and destination GPS coordinates were chosen as a route that we are familiar with, so we could immediately recognize that the generated route was valid. For other route generation tests, the route was validated by comparing the generated directional commands to Google Maps.

Test 3: Conversion of Valhalla Output Into the 14-Byte Packet

The third test verified that the maneuver information obtained from the routing engine was correctly encoded into the custom 14-byte packet format. This included validating the conversion of maneuver type into direction codes, formatting distance into the two-byte representation, selecting the correct unit byte, and truncating or padding street names to fit the fixed-length packet. Successful operation was confirmed by comparing the encoded packet versus the expected maneuver information from the routing output.

Test 4: Bluetooth Connection From Python Using Bleak

The fourth test verified that the host computer could establish a Bluetooth Low Energy connection to the RN4871 module using Python. Bluetooth functionality had already been confirmed using Microchip's SmartData application, so this stage focused specifically on reproducing the connection in Python through the Bleak library. Successful operation was confirmed when the Python program was able to discover the device, connect to the correct RN4871 module, and maintain a stable BLE connection.



```
connect() -> False (10002.7 ms)
ENQ (KEEPALIVE): 4b 41 0a
Connect error:
Connected: True
Notify enabled on 49535343-1e4d-4bd9-ba61-23c647249616
TX DONE (KEEPALIVE) -> True (104.4 ms)
Notify enabled on 49535343-4c8a-39b3-2f49-511cff073b7e
ENQ (KEEPALIVE): 4b 41 0a
TX DONE (KEEPALIVE) -> True (3.5 ms)
```

From (lat, lon): 49.277684, -123.108586 To (lat, lon): 49.272365, -123.155302

No route loaded.

Connect BLE Fetch Route Send Next Command Disconnect

Figure 5.4.4 – Python Output Showing that a Bluetooth Connection was Successful

Test 5: End-to-End Packet Transmission to the Smart Glasses

The final software test verified full system integration. In this test, 14-byte navigation packets generated by the Python software were transmitted over BLE to the RN4871 module, forwarded over UART to the MSP430, and then used to update the OLED display. This demonstrated that the routing, packet encoding, Bluetooth communication, and embedded display update pipeline were all functioning together correctly. Successful operation was confirmed when the correct navigation graphics and text appeared on the OLED in response to transmitted software packets.

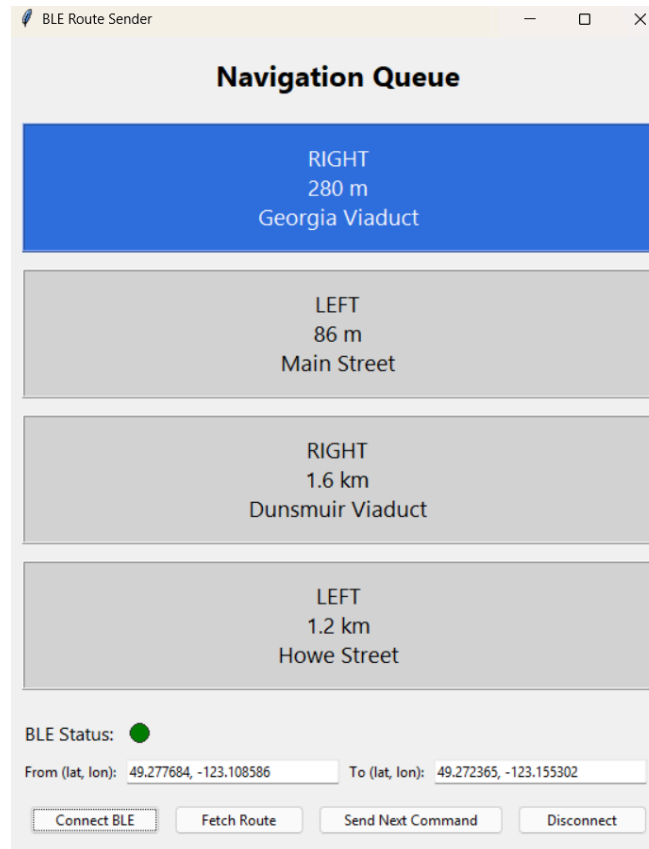


Figure 5.4.5 – Fully Implemented User Interface, including Route Generation, Byte-Packet Creation, and Bluetooth Connection

6. Functional Component #3 - Optics

6.1 Approach and Design

The objective of this functional component is to make the image produced by the mini OLED display visible and clear to one of the user's eyes. This functional component consists of two main parts: the lens and the reflector.

The lens design is critical because it sets the aperture angle and wavefront curvature of the light entering the eye. The human eye has a limited amount of optical power. If an object is too close to the eye, the wavefront curvature is too large, and the eye is unable to properly focus the image onto the retina, resulting in blur. In a smart glasses system, all components, including the display, are located very close to the user's eye, which makes this especially important. To address this, an optical system is required to reduce the divergence of the light emitted by the display and increase the apparent image distance. In our design, this is accomplished using a convex lens placed in front of the OLED. The lens reduces the wavefront curvature of the emitted light and increases the apparent size of the image, allowing the display to appear clearer and easier to view despite the small physical size of the micro-OLED screen.

The lens used in this design has a diameter of 8.8 mm and a focal length of 27 mm. It is positioned 26 mm from the micro-OLED display. The distance from the lens to the user's pupil is approximately 30 mm, including the folded optical path created by reflection from the semi-reflective film.

Without the lens, the optical setup effectively consists of only the OLED display and the eye separated by a distance of approximately 56 mm, as illustrated in Figure 6.1.1.

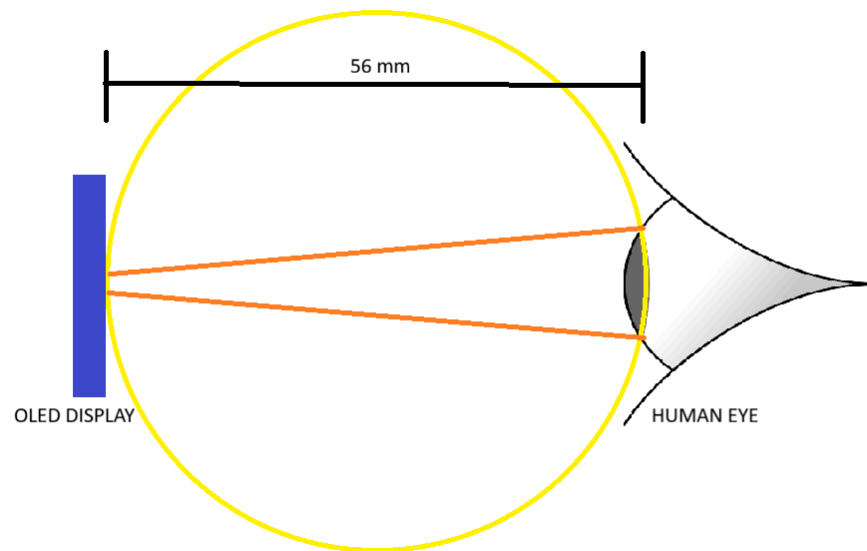


Figure 6.1.1 – Divergent rays emitted from the OLED display when viewed directly by the eye. Because the display is located only 56 mm from the pupil, the incoming wavefront has a small radius of curvature, requiring excessive accommodation and resulting in a blurred image.

The human eye cannot focus on objects that close. If you take a book and put it 56mm in front of your eyes, everything will be blurry and impossible to read. The inclusion of the convex lens modifies the optical system as shown in Figure 6.1.2.

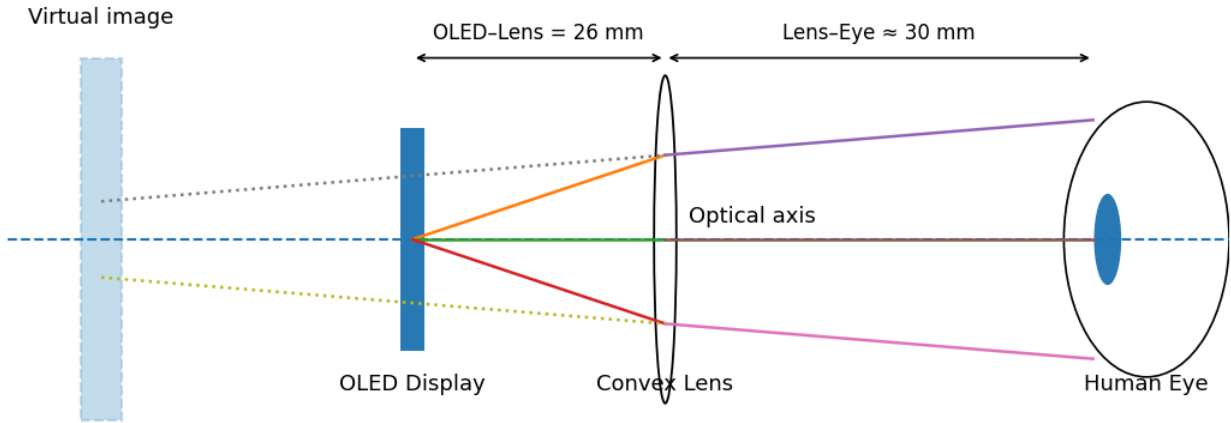


Figure 6.1.2 – Ray diagram of the OLED–lens optical system with the convex lens. Diverging light from the OLED passes through the convex lens and emerges with reduced divergence. Back-projection of the outgoing rays indicates a larger virtual image located farther from the eye than the physical display, increasing the apparent image distance and improving focus.

We can calculate the apparent distance of the virtual image using the thin lens equation:

$$\frac{1}{f} = \frac{1}{s} + \frac{1}{s'}$$

Where f is the focal length of the lens, s is the distance between the OLED and the lens, and s' is the apparent object distance. Plugging in our design parameters gives

$$\frac{1}{s'} = \frac{1}{27} - \frac{1}{26} \rightarrow s' = -702mm$$

The negative image distance indicates that the image is virtual, upright, and located on the same side of the lens as the OLED. Thus, the lens makes the display appear to originate from a point approximately 702mm in front of the lens. Since the user's eye is about 30 mm from the lens, the apparent image distance from the eye is approximately

$$d_{image-eye} = 702 + 30 = 732mm$$

This is dramatically further than the no-lens case and lies within a range that the human eye can accommodate much more comfortably. Therefore, the lens makes the OLED image significantly easier to focus on.

For the lens to function as intended, it must be properly aligned such that its optical axis is collinear with the normal of the OLED display. To ensure that the lens remains securely positioned, a custom lens holder was designed with three radial contact points and three axial contact points. The lens is then secured using

a retaining cap that is bonded to the lens holder. The lens holder design, highlighting the radial and axial locating features, is shown in Figure 6.1.3.

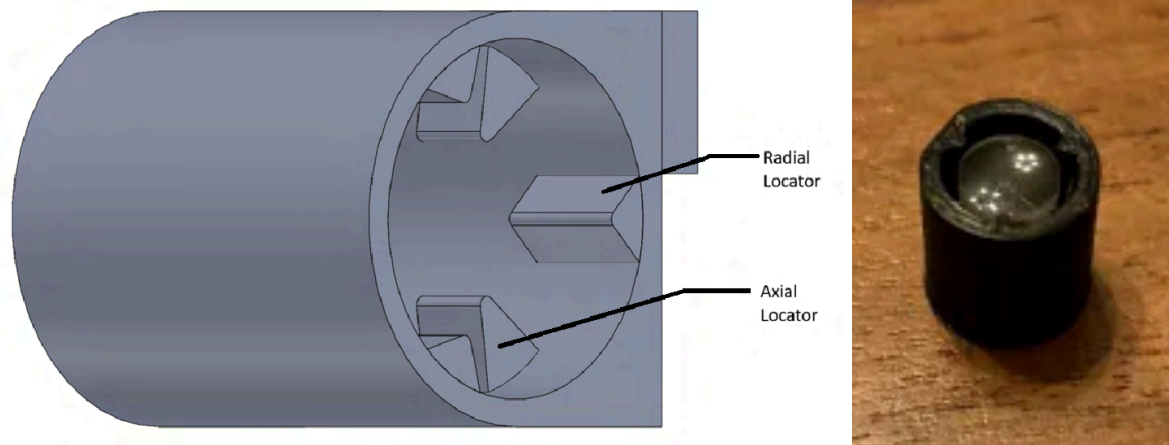


Figure 6.1.3 – Left: Lens holder design with radial and axial locating features. Right: 3D-printed proof-of-concept prototype. Lens cap not pictured.

The second component of the optical system is the reflector. In the smart-glasses design, the micro-OLED display faces forward, away from the user's eye. As a result, even if the image is properly conditioned by the lens, the light would not reach the eye without redirection. To address this, the optical system uses a folded-ray configuration in which a semi-reflective film redirects the image toward the user's eye.

A semi-reflective film was chosen so that the OLED image can be reflected into the eye while still allowing the user to see the surrounding environment through the lens. The lenses of the glasses were laser-cut from acrylic sheets, and the semi-reflective film was applied to the acrylic surface. The film adheres to the acrylic through electrostatic forces and therefore does not require additional adhesive.

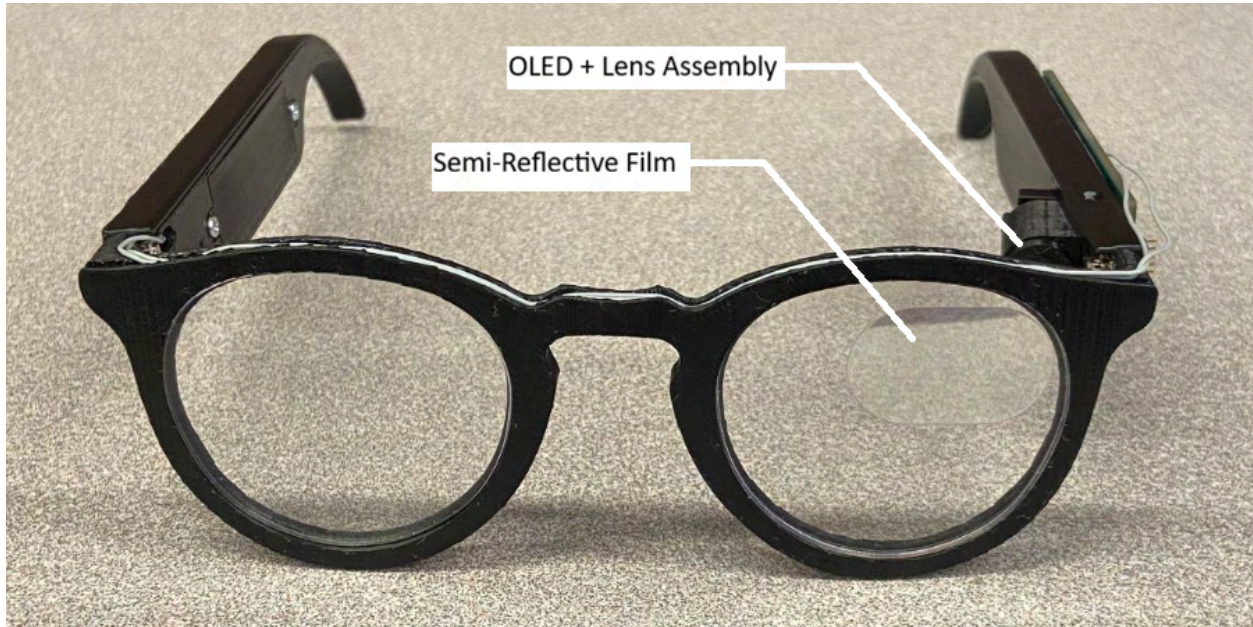


Figure 6.1.4 – Photograph of Completed Smart Glasses, with the Semi-Reflective Film and Optical Assembly Labelled

To ensure that the reflected image enters the eye, the angle between the OLED display and the reflective film needed to be properly set. This was achieved by fixing the angle of the OLED display within the left arm of the glasses. The design of this mounting configuration is described further in Section 7.

6.2 Inputs and Outputs

The input of this functional component is the light from the micro-OLED display, creating the image that feeds through the optical system. The inputs to the micro-OLED display could also be considered to be inputs for this FC, which are outlined in Table 8.2.1. The output is the light that emerges from the optical assembly toward the user's eye after reflection by the reflective film.

6.3 Parameters

The key parameters of this function are the optical distances, with the most important distance being the one between the screen and the lens. As shown in Section 6.1, this distance has the most significant impact on the apparent object distance, where it contributes 702mm to $d_{image-eye}$. The other 30mm comes from the remaining folded distance between the lens and the eye (which is the sum of the distance from the lens to the reflector and the reflector to the eye). We designed the assembly so that we can easily switch out the Lens Holder Assembly that is responsible for setting these distances, and also switch new lenses in without changing any other parts.

6.4 Testing and Results

The development process for the optical system followed the steps outlined below:

1. Design and build a mechanical system to hold the lens at a fixed angle while allowing linear motion along the optical axis for focusing.
2. Design a mechanical assembly that holds the OLED display in place and integrates the lens system described in Step 1.
3. Add a plane mirror to the assembly to emulate the folded path in the final product.
4. Design and fabricate lenses for the glasses with an embedded reflective film on the left eye.
5. Perform full mechanical assembly of the glasses and validate that the image is visible and clear.

Figure 6.4.1 shows a pictorial overview of the optical system development process and the progression of assemblies.

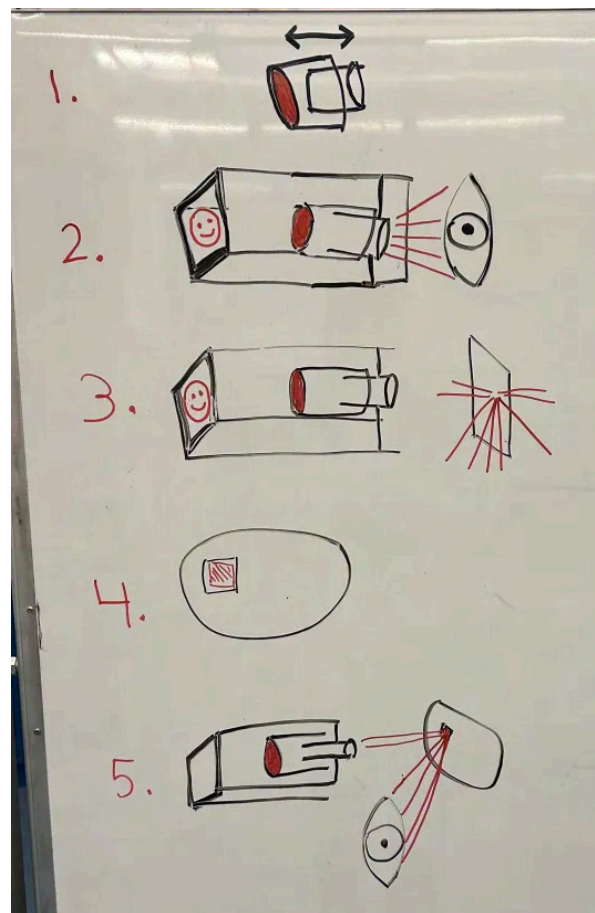


Figure 6.4.1 – Development Plan, Pictured

We completed steps 1-3 for the MCM demonstration and the later steps while developing FC4. Fig. 6.4.2 shows the image through the final product, which is clear and visible.



Figure 6.4.2 – User Perspective Looking into the Glasses During Final Testing

7. Functional Component #4 - Mechanical Design

7.1 Approach and Design

This FC is the design of the frame, which is like a normal pair of glasses that is modified to house the optics and electronics. Fig. 7.1.1 shows a labeled assembly with the parts that we designed and 3D printed.

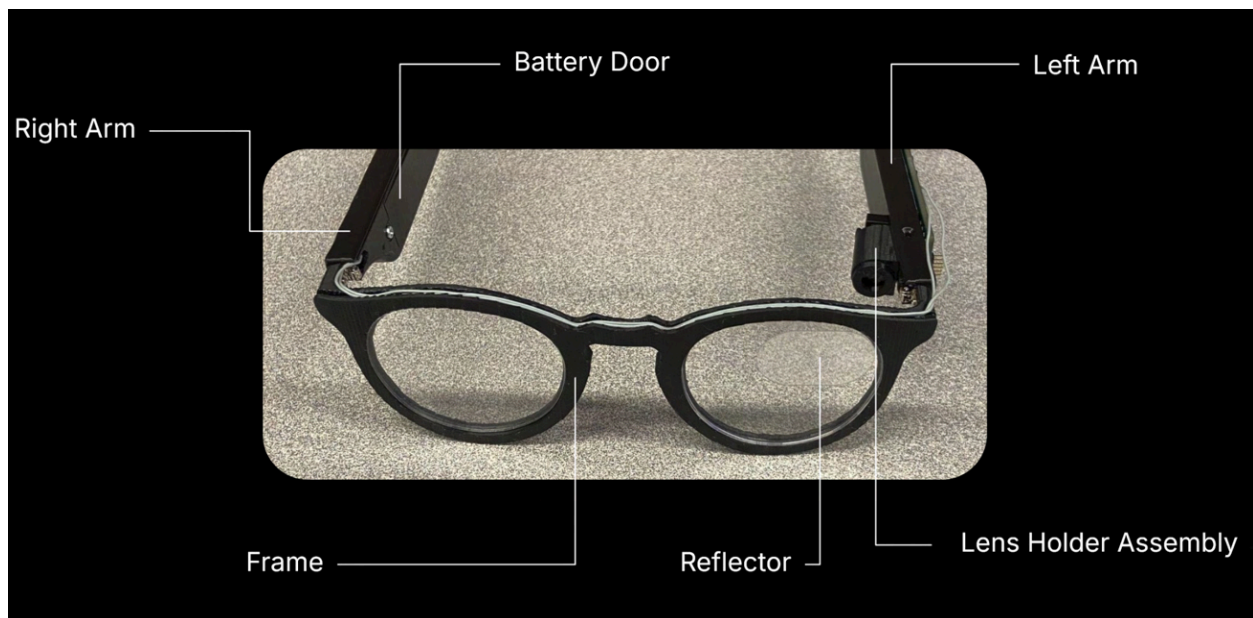
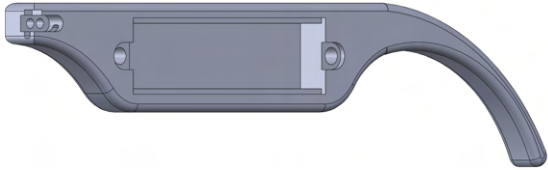
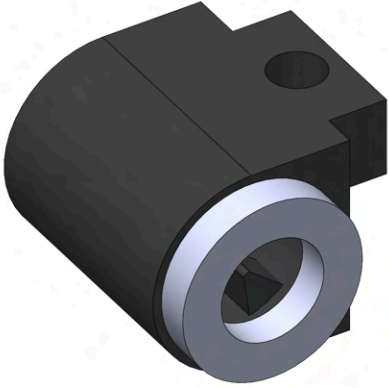
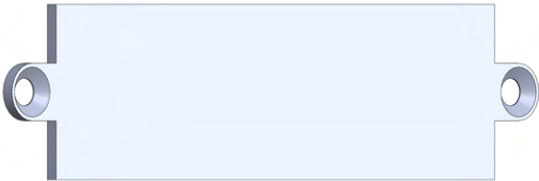
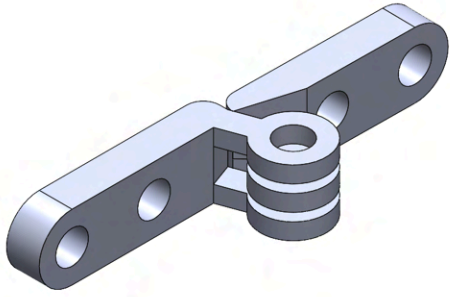
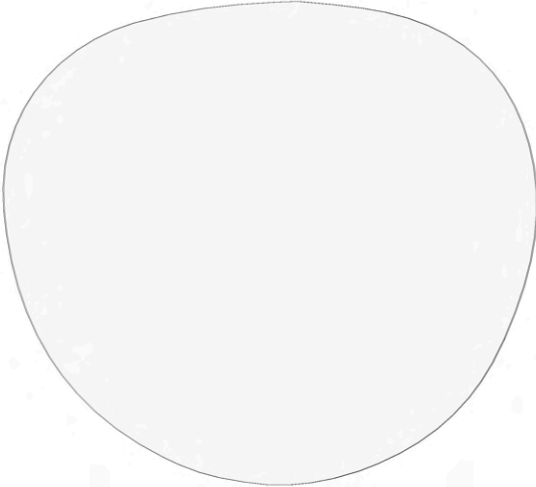


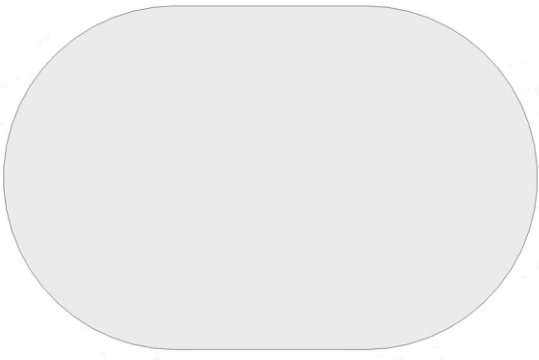
Figure 7.1.1 – Labeled Assembly

Table 7.1.1 gives a complete breakdown of the components in the assembly.

Table 7.1.1 - Mechanical Parts

	Right Arm The Right Arm has two heat inserts for the battery door and two heat inserts for the right hinge. There is a channel through the plastic for the battery wires. <i>Manufacturing: 3D Printing</i>
	Left Arm The Left Arm has two heat inserts for the left hinge and mating geometry for the Lens Holder Assembly. <i>Manufacturing: 3D Printing</i>
	Lens Holder Assembly The Lens Holder Assembly consists of two parts: the Lens Holder and the Cap, which retains the lens. It also serves the purpose of locking the screen in the Left Arm. The geometry is set such that the focal distance of the lens is the distance between the screen and the lens. This assembly promotes a modular design, where different lenses can be used by simply swapping a new Lens Holder into the Right Arm. <i>Manufacturing: 3D Printing</i>
	Frame The frame has four heat inserts for the hinges and a channel along the top for running the battery wires. Aside from the channel, the frame looks exactly like the frame of a standard pair of glasses. <i>Manufacturing: 3D Printing</i>

	Battery Door The battery door is fastened into the Right Arm with two M2 screws, and fixes the battery. <i>Manufacturing: 3D Printing</i>
	Hinge The hinges allow the glasses to fold, and also enabled us to print the assembly in separate parts without gluing, which gives better quality parts. <i>Manufacturing: Off-the-shelf</i>
	Eye Lens The Eye Lenses are about 2.36mm thick and press fit into the frame. They are laser cut out of an acrylic sheet. <i>Manufacturing: Laser Cut</i>

	Reflector The reflector is a slot with a 9mm radius and a 10mm centre-to-centre distance. It is laser cut out of a thin reflective sheet and allows the reflection from the screen to be observed as well as the surroundings behind it, giving a projecting effect. <i>Manufacturing: Laser Cut</i>
Misc. screws and heat inserts	M1 screws and inserts were used for the hinges and M2 screws and inserts were used for the battery door.

7.2 Inputs and Outputs

The input for this function could be considered to be the dimensions of the user's head, which determines the dimensions of the parts. Other inputs are constraints from other FCs such as the PCB dimensions and the optical system (FC3 and FC5).

7.3 Parameters

The primary dimensions for glasses are shown in Fig. 7.3.1, and were selected to fit well on our heads. Fortunately, our heads have similar dimensions and the pair that we ended up with fits us both. The dimensions of our glasses are 52-20-145.

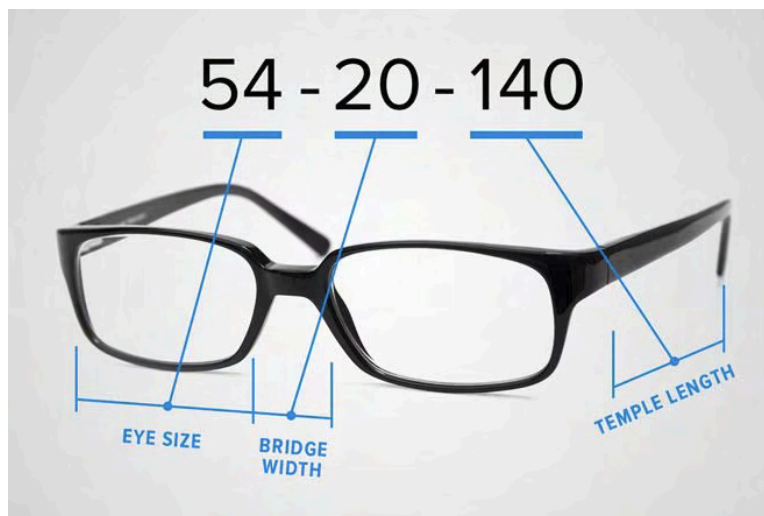


Figure 7.3.1 - Significant frame dimensions

(<https://www.allaboutvision.com/eyewear/eyeglasses/styles/glasses-frame-size/>)

The temple width is 20mm to fit the PCB, and the arms are 7.5mm thick to give enough material for feature recesses, wire routing, and threading.

Designing the frame and arms to fit our heads well (like our actual glasses) was one of the most challenging aspects of the mechanical design, and it took many iterations to arrive at the final product. The nose bridge features on the frame have a large impact on the comfort and wearability, and it took a few tries to get it right. Another mention is for the hinges, where we switched to the recessed hinge design so that the plastic supports them against the large couple moments that they experience.

7.4 Testing and Results

The mechanical design ended up being an enormous success because we think that we achieved the sleek appearance that we were aiming for. The glasses stay on our faces, fit comfortably, and wear like normal glasses. Fig. 7.4.1 shows what it looks like to look into the glasses, where you can see the text floating in your field of vision, and also see the surroundings in the background. That is, no FOV is blocked off like it would be if a solid mirror reflector were to have been used.

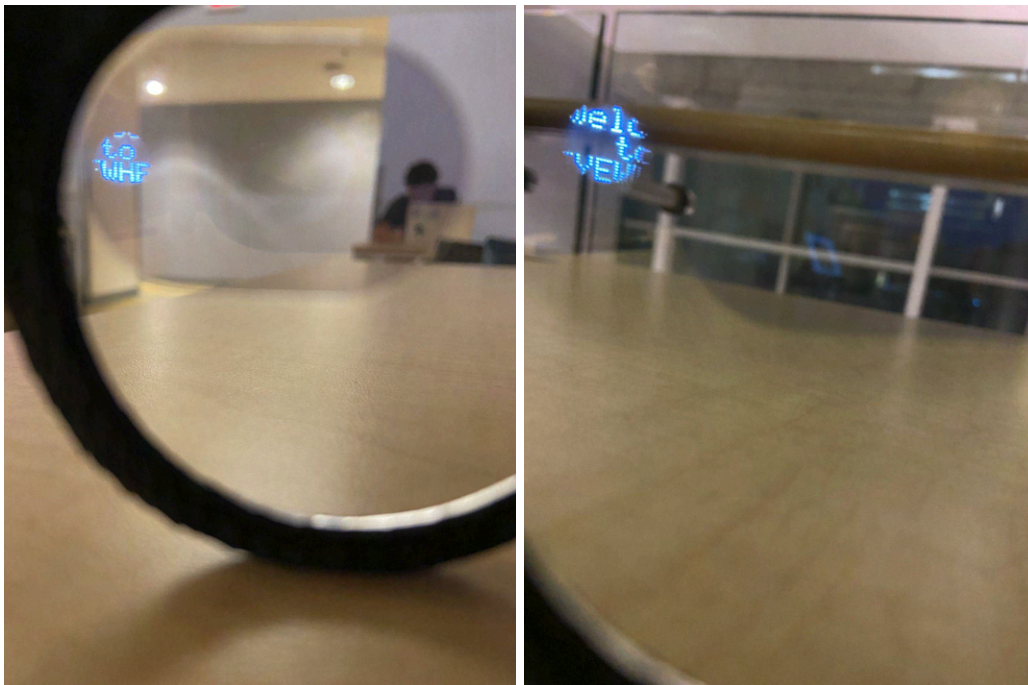


Figure 7.4.1 - Significant frame dimensions

8. Functional Component #5 - Electronics Design

8.1 Approach and Design

This FC includes the PCB, which facilitates all of the functionality needed from FC1. The system runs on a small 110mAh 3.7V Li-ion battery, so the PCB has a linear regulator to provide a 3V3 rail for the MCU

and the wireless module. The screen is powered with the battery voltage and uses its internal boost converter to get 7.5V. Fig. 8.1.1-4 shows the full schematic for the PCB.

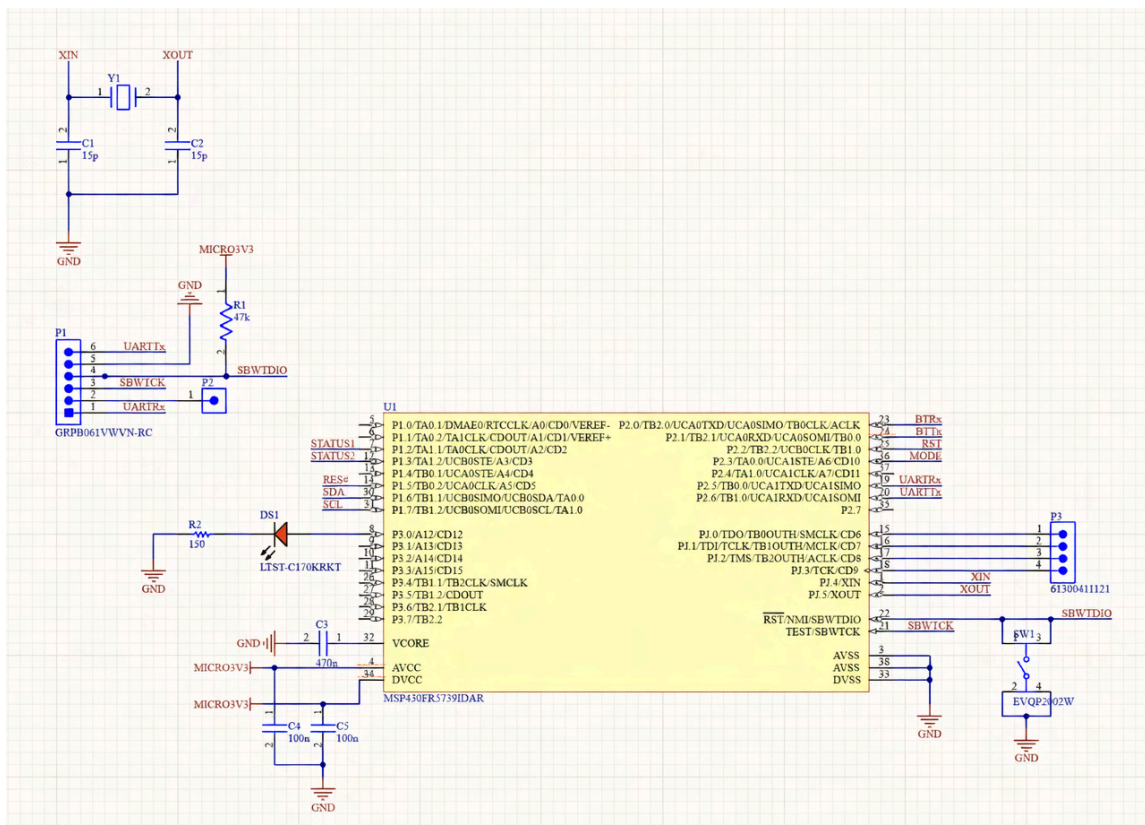


Figure 8.1.1 – Electrical Schematic — MCU Page

The MCU page has the MPS430 and its peripherals, including the external 32.768kHz XTAL and the Spy-Bi-Wire interface for flashing. Another header exposes the JTAG pins, and there is an extra LED for testing and debugging.

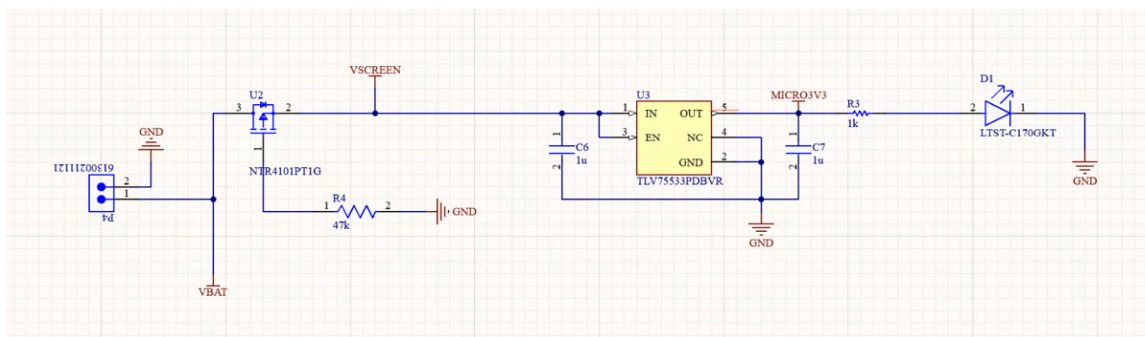


Figure 8.1.2 – Electrical Schematic — Power Rail Page

The power page has the TLV75533PDBVR linear regulator with reverse polarity protection at the input and a power status LED.

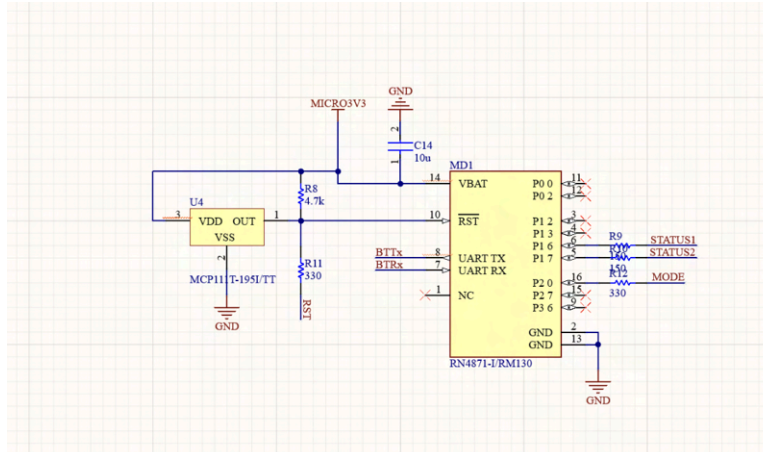


Figure 8.1.3 – Electrical Schematic — Bluetooth Module Page

The bluetooth page has the RN4871 module and the supervisor module for the 3V3 rail. STATUS1 and STATUS2 go to GPIOs on the MCU and communicate the state of the bluetooth connection, and the voltage at MODE during reset determines the application mode.

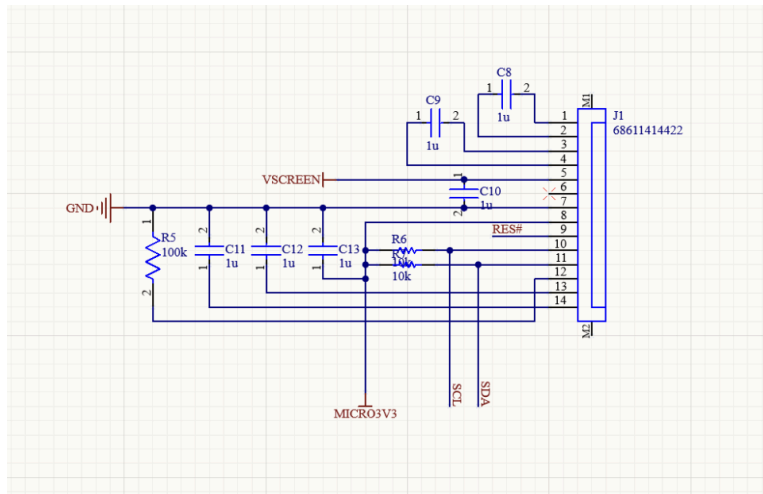


Figure 8.1.4 – Electrical Schematic — Screen Interface Page

The screen interface page includes all of the necessary components for the screen driver. There are decoupling capacitors, flying boost capacitors, I2C pullups, and a resistor for the current reference pin.

Before proceeding to layout we breadboarded everything to make sure that there were no issues at the schematic level. Fig. 8.1.5 shows the PCB layout, where the main focus in placing the components revolved around getting the board outline to be long and slender to mount to an arm.

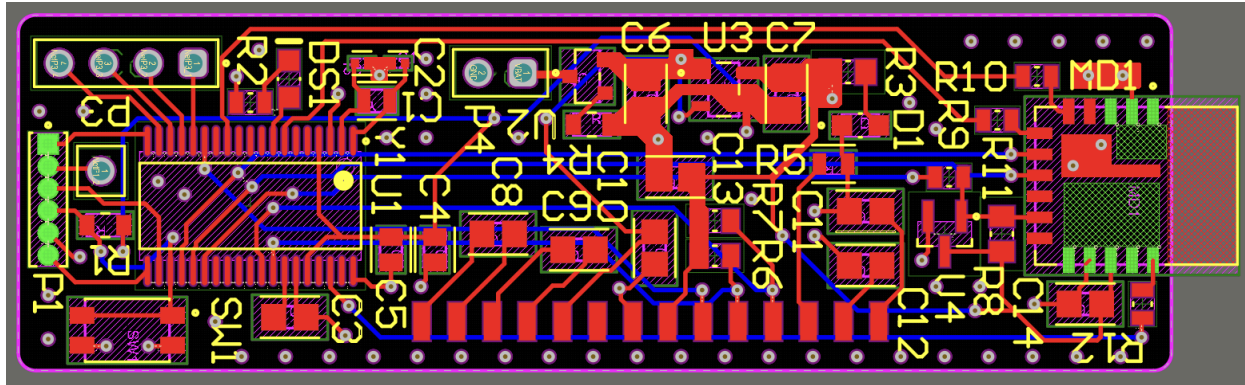


Figure 8.1.5 – PCB Layout

8.2 Inputs and Outputs

The inputs of the PCB are the DC voltage from the battery and the packets over the bluetooth link, and the outputs are all of the signals shown in Table 8.2.1 for driving the screen.

Table 8.2.1 - Display pinout from datasheet

(https://www.buydisplay.com/download/manual/ER-OLED0.32-1_Datasheet.pdf)

Pin No	Pin Name	Description
1~2	C2P/C2N	Positive Terminal of the Flying Inverting Capacitor Negative Terminal of the Flying Boost Capacitor The charge-pump capacitors are required between the terminals. They must be Floated when the converter is not used.
3~4	C1P/C1N	Positive Terminal of the Flying Inverting Capacitor Negative Terminal of the Flying Boost Capacitor The charge-pump capacitors are required between the terminals. They must be Floated when the converter is not used.
5	VBAT	Power Supply for DC/DC Converter Circuit This is the power supply pin for the internal buffer of the DC/DC voltage converter. It must be connected to external source when the converter is used. It should be Connected to VDD when the converter is not used.
6	VBREF	NC
7	VSS	Ground of Logic Circuit This is a ground pin. It acts as a reference for the logic pins. It must be Connected to external ground.
8	VDD	Power Supply for Logic This is a voltage supply pin. It must be connected to external source.
9	RES#	Power Reset for Controller and Driver This pin is reset signal input. When the pin is low, initialization of the chip is Executed. Keep this pin pull high during normal operation.
10	SCL	IIC Bus Clock Signal The transmission if information in the I2C bus is following a clock Signal. Each Transmission of data bit is taken place during a single clock period of this pin.
11	SDA	I2C Bus Data Signal This pin acts as a communication channel between the transmitter and the Receiver.
12	IREF	Current Reference for Brightness Adjustment This pin is segment current reference pin. A resistor should be connected Between this pin and VSS. Set the current at 12.5A maximum.
13	VCMH	Voltage Output High Level for COM Signal This pin is the input pin for the voltage output high level for COM signals. A capacitor should be connected between this pin and VSS.
14	VCC	Power Supply for OEL Panel This is the most positive voltage supply pin of the chip. A stabilization capacitor Should be connected between this pin and VSS when the converter is used. It Must be connected to external source when the converter is not used.

Pins 1-2 and 3-4 have charge-pump capacitors across them to use the built-in DC-DC converter, which eliminates the need for a boost converter and another power rail on the PCB. Flashing is done over the Spy-Bi-Wire interface, which is much more compact than a USB flashing interface.

8.3 Parameters

The key parameters of this PCB are summarized in Table 8.3.1.

Table 8.3.1 - PCB parameters

Key Parameter	Information
Voltages	3.7V from battery and 3.3V on power rail after the linear regulator
Dimensions	65mm x 20mm x 1.6mm
MCU	MSP430FR5739
Wireless Module	RN4871

One parameter that we adjusted was the I2C pullup resistors and transmission rate, because with longer wire lengths some bits were being lost due to the RC nature of the edges.

8.4 Testing and Results

When we received the PCBs we tested power and flashing with simple firmware to blink the LED. After those tests were passed we connected the screen and made sure that firmware for writing data to the screen worked correctly. We tested the latter in two stages:

1. Flash firmware that writes images to the screen.
2. Use wireless communication and check that packages are received and everything works as intended.

One of the biggest challenges during PCB bring-up was soldering an extension to the screen. The FPC connector coming out of the screen uses a unique 0.62mm pitch, which is not compatible with any connectors or extenders available for purchase. Fig. 8.4.1 shows the first successful soldering attempt that we made, which worked for our first prototype. In the final version we found a more efficient way to solder the screen to the PCB that made the wiring much cleaner, which is shown in Fig. 8.4.2.

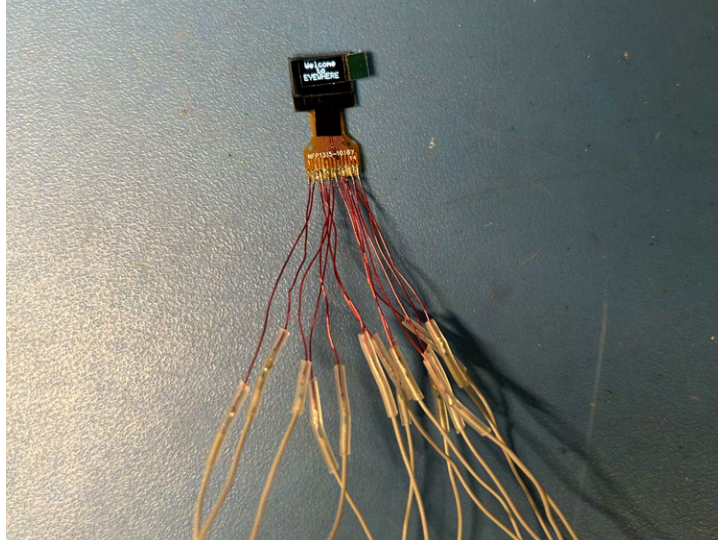


Figure 8.4.1 - First Successful Screen Wiring

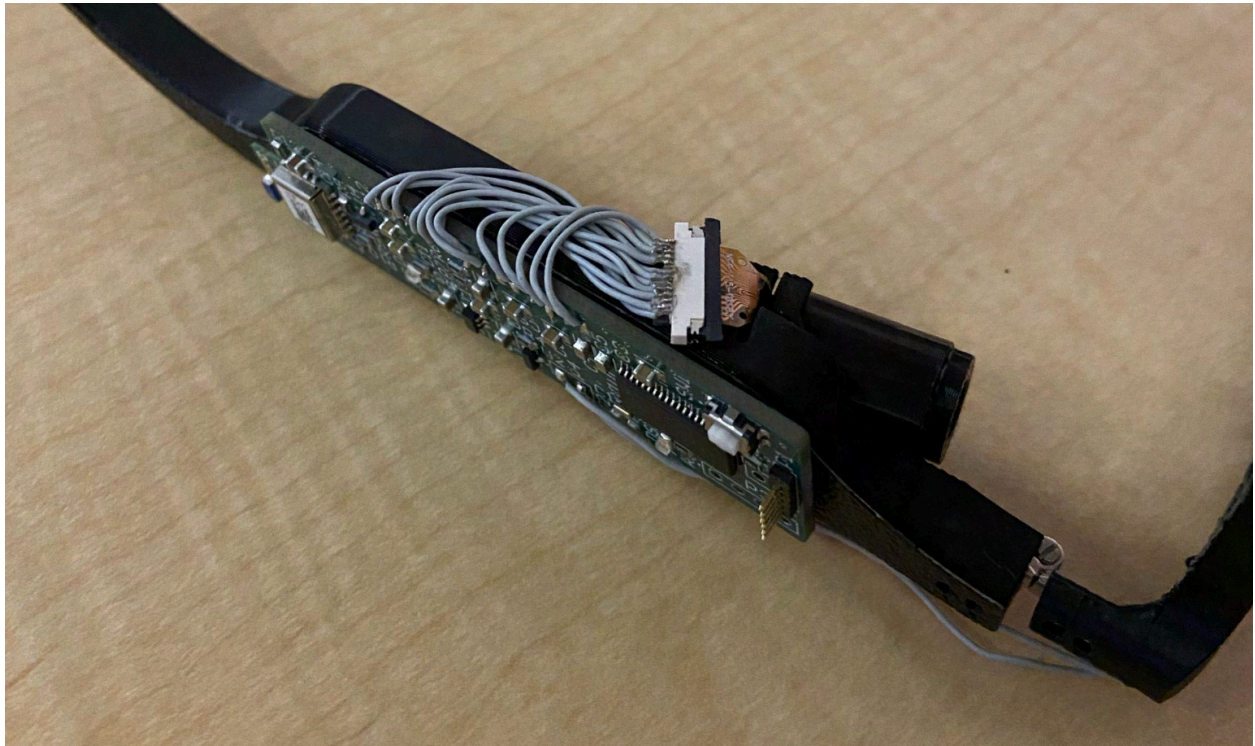


Figure 8.4.2 – Final Screen Wiring

The PCB functioned 100% as intended, and Fig. 8.4.3 shows a picture of the final product with the PCB.

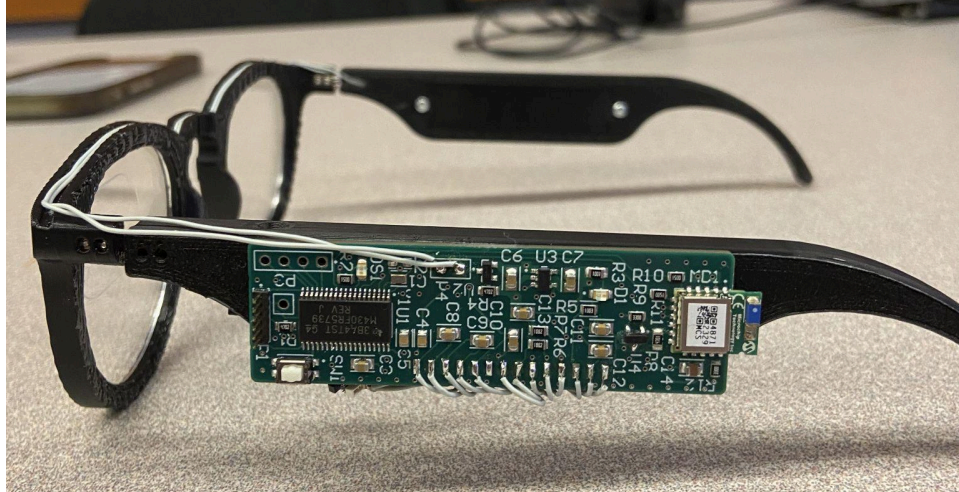


Figure 8.4.3 – Side View of Glasses

9. System Evaluation

In our earlier stages of development we used the MSP-EXP430FR5739 with a breadboarded circuit to interface with the screen for firmware development. This is the testing environment that we used to figure out how to configure and write to the screen, which we wrote bit-banged I2C firmware to do by following the driver datasheet. The development involved using the oscilloscope to watch the SDA and SCL lines to check for the correct sequences and acknowledgement while figuring out the correct sequences for startup and shutdown in the two operating modes: external 7.5V power supply or external 3.7V power supply with internal boost converter. After we could write to the screen we wrote additional functions to parse 14-byte data packets that we planned in the proposal and tested the system by sending packets over UART from a windows form that we developed (Fig. 9.1).

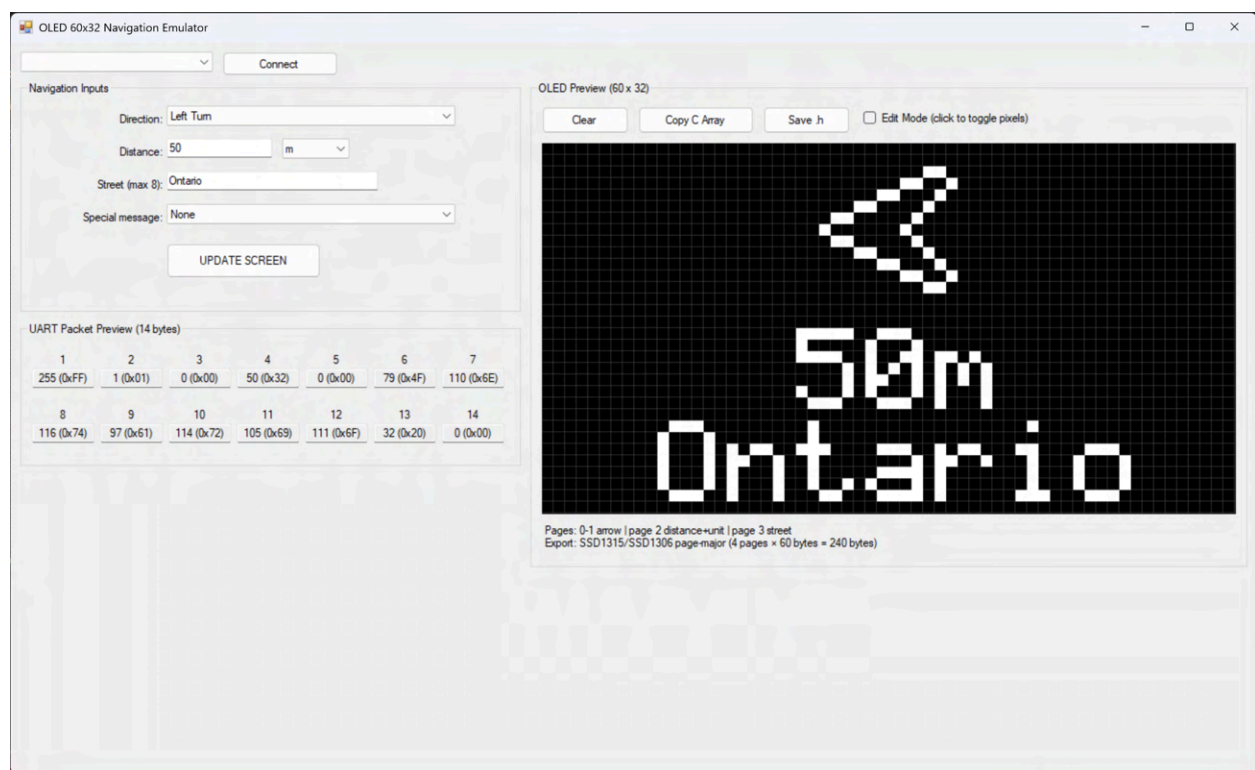


Figure 9.1 – Windows Form for Testing Firmware

Our polished aspect declared in the proposal was firmware and electronics, and this test setup was used for the firmware evaluation. We subsequently added the bluetooth module to this circuit using its development board, which enabled us to write and test the firmware for setting up the BLE connection and receiving forwarded data on the MSP430. Full firmware tests were conducted once these two parts were completed, and we were able to send data from our phones or laptops over the bluetooth link and it would be forwarded to the MSP430 from the BLE module. When we sent data in the 14-byte form it wrote to the screen, which satisfies all of the requirements for our firmware.

One issue that we came across when we soldered longer wires to the screen was that sometimes when we wrote data the picture was corrupted, meaning that the I2C data filling the display registers was corrupted.

To fix this we experimented with different pullup resistors and transmission rates to lower the RC time constant and decrease the edge frequency, eventually settling on a combination that gave reliable results. The wire length in the final product ended up being short (Fig. 8.4.2), and we could increase the write speed again to a new optimised value. This is in relation to the framerate of the screen, which is about 1FPS without tearing in the current version. We could conduct further optimisation to increase the framerate, but we deemed that it is not important for the performance of the product.

The PCB that was designed based on our test circuits worked as intended, with a final surface area of 13cm² with all of the components on the top layer. In the proposal we said that we were shooting for 5cm², but the size of the PCB ended up working out with the design. The average power draw is 7mA, and with our 110mAh battery the projected battery life is 15 hours. We have not verified this battery life because it has yet to die, but we have run it for about three hours so far without any issues.

For the mechanical design our metrics for success were the styling, form factor, fit, and durability. We think that the final product exceeded our expectations in all of those metrics, especially with regard to the form factor. The glasses stay on our heads without discomfort, and the ability to fold them is another bonus that we managed to include.



Figure 9.2 - Final Product

In our proposal we outlined final tests that we would conduct, titled “Full System Performance Testing”. These involved walking, biking, and driving routes where we could test the instructions updating at the correct time, the wearability in real applications, and the bluetooth connection range. We have not formally conducted these tests yet, but are confident that we are at the point where the glasses would pass them.